

Custom and Multiple Exception Handling

Q1.

Create a custom exception class called `OutOfStockException` for a Flipkart-style shopping cart, which should extend `Exception` and display a message when the requested quantity of a product exceeds available stock.

Answer:

```
class OutOfStockException extends Exception {
    public OutOfStockException(String productName, int requested, int available) {
        super("Out of stock: requested " + requested + " unit(s) of \"" + productName +
            "\"" but only " + available + " available.");
    }
}

public class FlipkartStockCheck {

    public static void placeOrder(String productName, int requested, int available) throws OutOfStockException {
        if (requested > available) {
            throw new OutOfStockException(productName, requested, available);
        }
        System.out.println("Order placed for " + requested + " unit(s) of " + productName);
    }

    public static void main(String[] args) {
        try {
            placeOrder("Wireless Mouse", 5, 10);
            placeOrder("Mechanical Keyboard", 8, 3);
        } catch (OutOfStockException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

/* Sample Output:
Order placed for 5 unit(s) of Wireless Mouse
Error: Out of stock: requested 8 unit(s) of "Mechanical Keyboard" but only 3 available.
*/
```

Q2.

Write a Java method `processPayment` in a class called `PaymentHandler` that simulates a PhonePe payment. Use multiple catch blocks to handle `NumberFormatException` (for invalid amount input), `InsufficientFundsException` (if balance is too low), and a generic `Exception` for any other errors. Print a different error message for each case.

Answer:

```
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public class PaymentHandler {
```

```

private static double balance = 500.0;

public static void processPayment(String amountStr) {
    try {
        double amount = Double.parseDouble(amountStr); // may throw NumberFormatException

        if (amount > balance) {
            throw new InsufficientFundsException("Balance too low for this PhonePe payment.");
        }

        balance -= amount;
        System.out.println("Payment of Rs. " + amount + " successful. Remaining balance: Rs. " + balance);

    } catch (NumberFormatException e) {
        System.out.println("Error: Invalid amount entered. Please enter a valid number.");
    } catch (InsufficientFundsException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Error: Something went wrong - " + e.getMessage());
    }
}

public static void main(String[] args) {
    processPayment("200"); // valid payment
    processPayment("abc"); // invalid number
    processPayment("900"); // insufficient funds
}

/* Sample Output:
Payment of Rs. 200.0 successful. Remaining balance: Rs. 300.0
Error: Invalid amount entered. Please enter a valid number.
Error: Balance too low for this PhonePe payment.
*/

```

Q3.

Simulate a Zomato order flow where a user tries to place an order. Throw and handle multiple exceptions: `OutOfStockException` if the dish is unavailable, `InsufficientFundsException` if wallet balance is too low, and `IllegalArgumentException` if the order amount is negative. Ensure each exception is caught separately and prints a user-friendly message.

Answer:

```

class OutOfStockException extends Exception {
    public OutOfStockException(String message) { super(message); }
}

class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) { super(message); }
}

public class ZomatoOrderFlow {

    public static void placeOrder(boolean dishAvailable, double walletBalance, double orderAmount)
        throws OutOfStockException, InsufficientFundsException {

        if (orderAmount < 0) {
            throw new IllegalArgumentException("Order amount cannot be negative.");
        }
        if (!dishAvailable) {

```

```

        throw new OutOfStockException("Sorry, this dish is currently out of stock.");
    }
    if (orderAmount > walletBalance) {
        throw new InsufficientFundsException("Insufficient wallet balance to place this order.");
    }

    System.out.println("Order placed successfully for Rs. " + orderAmount);
}

public static void tryOrder(boolean dishAvailable, double walletBalance, double orderAmount) {
    try {
        placeOrder(dishAvailable, walletBalance, orderAmount);
    } catch (OutOfStockException e) {
        System.out.println("Order failed: " + e.getMessage());
    } catch (InsufficientFundsException e) {
        System.out.println("Order failed: " + e.getMessage());
    } catch (IllegalArgumentException e) {
        System.out.println("Order failed: " + e.getMessage());
    }
}

public static void main(String[] args) {
    tryOrder(true, 500, 300);    // success
    tryOrder(false, 500, 300);   // out of stock
    tryOrder(true, 200, 300);    // insufficient funds
    tryOrder(true, 500, -50);    // invalid amount
}

/* Sample Output:
Order placed successfully for Rs. 300.0
Order failed: Sorry, this dish is currently out of stock.
Order failed: Insufficient wallet balance to place this order.
Order failed: Order amount cannot be negative.
*/

```

Q4.

Given the following buggy code snippet for a ticket booking system, debug and fix it so that it correctly throws and catches `InsufficientFundsException` and prints the right error message when the user's balance is less than the ticket price. Hint: Make sure the `InsufficientFundsException` class provides a meaningful message and is properly caught.

Answer:

```

// ---- Buggy code given ----
// try {
//     int userBalance = 100;
//     int ticketPrice = 150;
//     if (userBalance < ticketPrice) {
//         throw new InsufficientFundsException();    // BUG: class doesn't exist / no message
//     }
//     System.out.println("Booking successful!");
// } catch (Exception e) {
//     System.out.println("Error: " + e.getMessage());    // BUG: getMessage() would return null
// }

// ---- Fixed code ----
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);    // pass a meaningful message to the parent Exception class
    }
}

```

```

    }
}

public class TicketBookingFix {
    public static void main(String[] args) {
        try {
            int userBalance = 100;
            int ticketPrice = 150;

            if (userBalance < ticketPrice) {
                throw new InsufficientFundsException(
                    "Insufficient balance: available Rs. " + userBalance +
                    ", ticket price Rs. " + ticketPrice);
            }
            System.out.println("Booking successful!");

        } catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

/* Sample Output:
Error: Insufficient balance: available Rs. 100, ticket price Rs. 150
*/
// Fixes made:
// 1. Defined the InsufficientFundsException class (it didn't exist before).
// 2. Gave it a constructor that passes a real message to super(), so getMessage() works.
// 3. Caught the specific InsufficientFundsException instead of a bare, message-less Exception.

```